Project Report

Project Title: Distributed Trip-Level Fuel Consumption Forecasting Using Large-Scale Vehicle Telemetry

Team Members:
Andrei Zhdanov (an.zhdanov@innopolis.university)
Ilyas Galiev (il.galiev@innopolis.university)
Mikhail Sofin (m.sofin@innopolis.university)
Kamilya Shakirova (k.shakirova@innopolis.university)

May 12, 2026

1 Introduction

In this project, we built a distributed pipeline for analyzing and forecasting trip-level fuel consumption. The source data comes from the Vehicle Energy Dataset (VED), where raw telemetry is stored at sensor-event level. This means that one vehicle trip consists of many timestamped rows with speed, location, engine, fuel, temperature, HVAC, and battery-related signals. Our goal was to convert this high-volume raw telemetry into trip-level analytical records and then use them to study and predict fuel consumption.

The project follows the main stages of a big data workflow: data collection, loading into PostgreSQL, importing to HDFS with Sqoop, profiling and EDA in Spark, trip-level feature engineering, benchmarking, Spark ML modeling, and dashboard creation in Apache Superset. We chose this structure because the raw table contains more than 22 million telemetry rows, so processing everything manually or locally would be inconvenient and slow.

The prediction target is `fuel_l_per_100km`, which describes trip-level fuel consumption in liters per 100 kilometers. We selected this target because it is easy to interpret in vehicle analytics: it measures consumption per distance rather than fuel usage per time. The dashboard presents the project in three logical tabs: Data Description, EDA, and PDA.

2 Business Objectives

The practical motivation of the project is fuel-efficiency monitoring for a vehicle fleet. If an organization operates many vehicles, fuel consumption becomes one of the important operational costs. Vehicle telemetry can help understand when fuel usage becomes high and which technical or driving-related factors are connected with inefficient trips.

The main business objectives are:

- to build a reproducible distributed pipeline for large-scale vehicle telemetry;
- to separate raw sensor-level data from trip-level analytical data;
- to identify data-quality limitations, especially missing fuel and battery fields;
- to analyze how fuel behavior changes by vehicle type, speed range, driving mode, temperature, HVAC usage, and engine displacement;
- to create a trainable trip-level dataset for forecasting `fuel_l_per_100km`;
- to compare distributed regression models and select the best one based on RMSE, MAE, and R2;
- to present the results in a Superset dashboard that explains the project results for both technical and non-technical users.

These objectives explain why the project includes both engineering and analytical stages. Before EDA and ML can produce useful results, the raw data has to be stored, checked, transformed, and aggregated correctly.

3 Data Description

The project uses two main data sources from VED:

- raw trip telemetry CSV files, where each row is a timestamped sensor record inside a trip;
- static vehicle metadata spreadsheets, which describe vehicle type, class, transmission, drivetrain, weight, engine type, engine displacement, and engine configuration.

The relational source layer contains two PostgreSQL tables:

- `vehicles`: one row per vehicle, used as static metadata;
- `trips`: one row per telemetry timestamp, used as the high-volume sensor table.

We separated these tables intentionally. Vehicle attributes are repeated conceptually across many trips, so storing them in a separate `vehicles` table avoids duplicating static metadata in the raw telemetry table. The `trips` table stays focused on sensor-level observations. Later, Spark joins these two tables by `vehid` to enrich telemetry rows with vehicle characteristics.

Important raw trip fields include `daynum`, `vehid`, `tripid`, `time`, `lat`, `long`, `speed`, `maf`, `rpm`, `abs_load`, `oat`, `fuel_rate`, HVAC fields, hybrid battery fields, and fuel trim fields. Important vehicle fields include `vehtype`, `vehclass`, `transmission`, `drive_wheels`, `gen_weight`, `eng_type`, `eng_dis`, and `eng_conf`.

4 Data Characteristics

The dataset is large enough to justify distributed storage and processing. Table 1 summarizes the main characteristics used in the report and dashboard.

Table 1: Main dataset characteristics.

Metric	**Value**
Raw telemetry rows	22,436,808
Vehicle metadata records	384
Unique vehicles in trips	384
Unique trips before final ML filtering	32,552
Trainable trips shown in dashboard	about 25.2k
Raw trip columns	22
Vehicle metadata columns	9
Main ML target	`fuel_l_per_100km`

The dataset contains several groups of variables. Driving behavior is represented by speed, stop-go ratio, idle time, and speed distribution statistics. Engine behavior is represented by MAF, RPM, absolute engine load, and engine displacement. Environmental

context is represented by outside air temperature and HVAC power. Vehicle context is represented by vehicle type, vehicle class, transmission, drivetrain, weight, engine type, and engine configuration.

The most important data-quality issue is missingness. Direct `fuel_rate` measurements are sparse, and hybrid battery fields are naturally sparse because they apply only to some powertrain types. Selected missingness values from the raw telemetry table are shown in Table 2.

Table 2: Selected missingness values in raw telemetry.

Feature	**Null count**	**Null share**
speed	1,396	0.01%
maf	4,178,670	18.62%
rpm	482,305	2.15%
abs_load	5,965,212	26.59%
oat	9,368,517	41.76%
fuel_rate	21,540,711	96.01%
air_cp_kw	21,542,533	96.01%
heater_power	21,558,450	96.09%
hv_battery_current	18,708,156	83.38%
hv_battery_soc	18,708,156	83.38%

These missing values affected our technical decisions. Since `fuel_rate` is missing for most rows, we could not rely only on the direct fuel-rate column. Instead, Stage 2 creates a unified estimated fuel-rate field using direct `fuel_rate` when available and a MAF-based approximation when it is missing.

5 Architecture of Data Pipeline

We designed the pipeline according to the course requirements: the data had to pass through relational storage, distributed storage, distributed processing, and final dashboard presentation. Figure 1 shows the architecture.

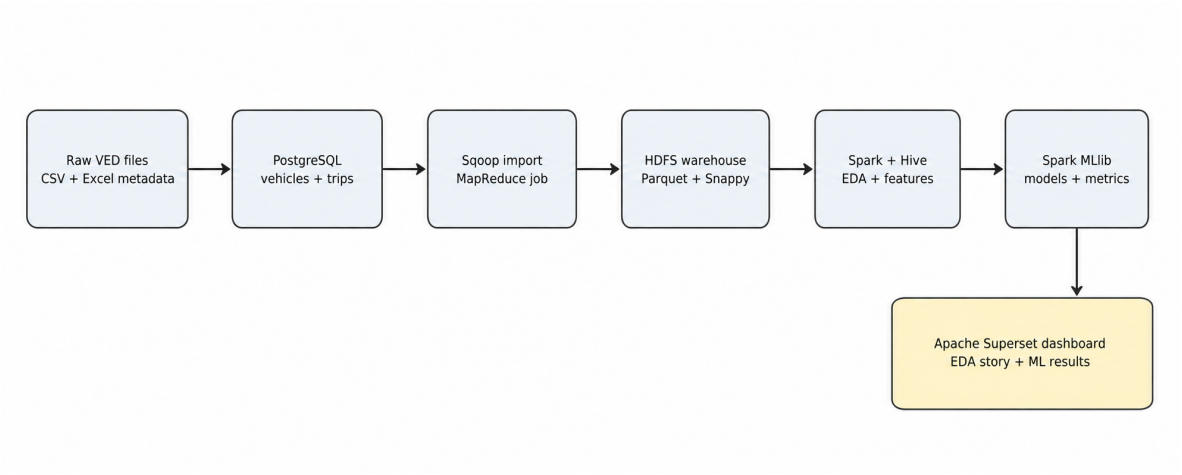

Figure 1: Pipeline architecture.

5.1 Input

The main inputs are raw VED telemetry CSV files, static vehicle metadata spreadsheets, SQL scripts for PostgreSQL, Python/Spark scripts for processing, and secret files for cluster authentication. The raw dataset is not stored in GitHub because it is large. Instead, it is downloaded or kept on the cluster and processed by scripts.

5.2 Processing Stages

The project is organized into four major stages, with additional profiling and benchmarking steps. Table 3 describes the input, processing, and output of each stage.

Table 3: Input, processing, and output of each project stage.

Stage	**Input**	**Processing**	**Output**
Stage 1	Raw CSV telemetry files and vehicle metadata spreadsheets	Prepare source data, audit files, create PostgreSQL schema, load `vehicles` and `trips` tables, benchmark ingestion, and import data to HDFS using Sqoop	PostgreSQL `vehicles`/`trips` tables; HDFS warehouse in Parquet + Snappy format; data audit and ingest benchmark outputs

Stage	Input	Processing	Output
Stage 2	HDFS Parquet warehouse	Profile raw warehouse tables, analyze column types, missingness, numeric summaries and quantiles, join vehicle metadata with trip telemetry, perform Spark EDA, approximate fuel rate, aggregate records to trip level, and export chart-ready data	Profiling outputs, missingness tables, numeric summaries, EDA tables, local CSV exports, and trip-level feature table
Stage 3	Stage 2 trip-level features	Filter invalid and unrealistic trips, remove outliers, select features using mutual information, create train/test datasets, preprocess features in Spark ML, train regression models, tune hyperparameters with cross-validation, and evaluate predictions	Feature-selection table, train/test Parquet datasets, Linear Regression, Random Forest, and GBT models, prediction outputs, evaluation metrics, and best-model metadata
Stage 4	Local CSV artifacts from previous stages	Upload dashboard-ready files to HDFS, create external Hive tables, validate dashboard data sources, and connect the prepared tables to Apache Superset	`stage4_*` Hive tables, dashboard-ready datasets, and Superset dashboard data layer

5.3 Output

The most important output locations are:

- `/user/team15/project/warehouse`: raw PostgreSQL tables imported to HDFS;
- `/user/team15/project/profiling`: profiling results for raw HDFS tables;
- `/user/team15/project/analytics`: Stage 2 analytical Parquet outputs;
- `/user/team15/project/analytics/trip_features`: ML-ready trip-level feature table;
- `/user/team15/project/ml_prepared`: feature-selected train/test datasets;
- `/user/team15/project/models`: trained Spark ML models;

- `/user/team15/project/output`: model evaluation, predictions, CV results, and best-model metadata;
- `/user/team15/project/stage4`: CSV folders exposed to Hive/Superset;
- Apache Superset dashboard: final presentation layer.

6 Data Preparation

Data preparation was one of the most important parts of the project because the raw telemetry could not be used for ML directly. One trip is represented by many sensor rows, some important fields are missing, direct fuel measurements are sparse, and vehicle metadata is stored separately. For this reason, we designed the preparation process to move gradually from raw sensor data to clean trip-level records.

1. **Vehicle metadata preprocessing.** The vehicle metadata spreadsheets were converted into `vehicles.csv`. We did this because PostgreSQL and Spark work more reliably with clean CSV/Parquet inputs than with multi-sheet Excel files.
2. **PostgreSQL loading.** The cleaned `vehicles` table and raw `trips` table were loaded into PostgreSQL. We used PostgreSQL as the relational raw layer because it supports schema constraints and provides a stable input for Sqoop.
3. **Sqoop ingestion to HDFS.** Sqoop was selected because it is designed for bulk transfer from relational databases to HDFS. The `trips` table was imported with four mappers and split by `vehid` to parallelize the large table import.
4. **Parquet + Snappy storage.** The HDFS warehouse uses Parquet with Snappy compression. We chose Parquet because the workload is read-heavy and most analytical queries read only selected columns such as speed, RPM, MAF, vehicle type, or fuel fields. We used Snappy because it gives a good balance between compression and read speed, which was useful because the same data was read many times by Spark, Hive, and Superset.
5. **NaN and NULL handling.** Spark converts invalid floating-point NaN values into nulls. This makes missingness analysis more consistent and prevents hidden invalid values from entering feature engineering.
6. **Vehicle-trip enrichment.** Trip telemetry is joined with vehicle metadata by `vehid`. This is required because vehicle type, engine displacement, weight, and drivetrain are important explanatory variables.
7. **Fuel-rate approximation.** When direct `fuel_rate` is unavailable, Stage 2 estimates fuel rate from MAF:

$$\texttt{fuel_rate_est_lhr} = \frac{\texttt{MAF}}{14.7 \times 745} \times 3600$$

   where 14.7 is the gasoline stoichiometric air-fuel ratio, 745 g/L is the gasoline density used in the approximation, and 3600 converts seconds to hours.

8. **Trip-level aggregation.** Sensor-level records are aggregated into one row per `(vehid, tripid)`. This step is essential because the ML target is trip-level fuel consumption, not a single sensor timestamp.
9. **Filtering for ML.** Trips with invalid target values, very short distances, invalid durations, or unrealistic speed values are removed. The target is also filtered by approximate 1st and 99th percentiles to reduce the effect of outliers.

7 ER Diagram

The relational schema contains two main tables: `vehicles` and `trips`. One vehicle can have many telemetry records, so the relationship is one-to-many. The foreign key is `trips.vehid`, which references `vehicles.vehid`. We kept this schema intentionally simple because the PostgreSQL layer is used mainly as a clean relational source layer before HDFS ingestion.

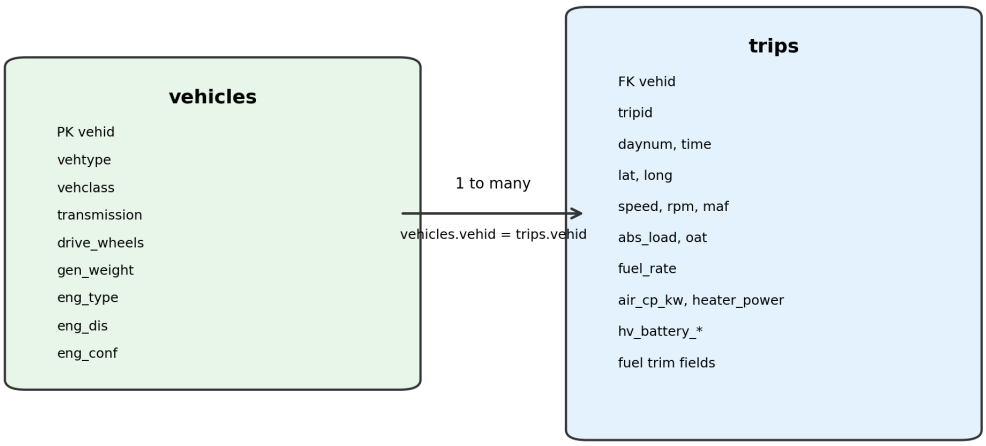

Figure 2: ER diagram of the PostgreSQL source schema.

8 Samples from the Database

Table 4 shows example rows from the `vehicles` table after metadata preprocessing.

Table 4: Sample rows from the vehicles table.

vehid	vehtype	vehclass	transmission	weight	eng. dis.	eng. conf.	
7	ICE	SUV	AUTOMATIC	NO DATA	4500	3.6	6-FI
8	ICE	Car	5-SP MANUAL	NO DATA	2500	1.5	4-FI

Table 5 shows examples from the trip-level feature table. These are not raw telemetry rows; they are aggregated records used for analysis and ML.

Table 5: Sample trip-level records used for ML.

vehid	tripid	duration min	distance km	speed mean km/h	speed p95 km/h	fuel used L	L/100km	type
587	120	71.13	133.86	73.66	117.00	11.81	8.82	ICE
108	1494	81.92	132.31	71.58	117.00	16.07	12.15	ICE
12	705	70.59	132.01	79.32	120.00	14.28	10.81	ICE
465	956	123.43	130.97	57.96	108.00	13.75	10.50	ICE

9 Creating Hive Tables and Data Preparation

Hive-compatible tables are used to expose HDFS outputs to Superset and SQL-based analysis. Stage 2 creates Parquet analytical outputs such as missingness, vehicle distribution, speed-bin analysis, driving-mode analysis, HVAC analysis, engine-displacement analysis, geographical activity, and the trip-level feature table. These outputs are stored under `/user/team15/project/analytics`.

Stage 4 creates external Hive tables with the prefix `stage4_*`. These tables point to dashboard-ready CSV folders under `/user/team15/project/stage4`. Its purpose is to prepare stable Hive tables, so that the dashboard charts can read data from fixed and reproducible sources.

The most important Hive/Superset tables are:

- `stage4_data_audit`: local raw-file audit;
- `stage4_table_overview`: row and column counts for warehouse tables;
- `stage4_missingness`: column-level missingness;
- `stage4_numeric_summary` and `stage4_numeric_quantiles`: descriptive statistics;
- `stage4_feature_selection`: feature scores and selected features;
- `stage4_train_test_metadata`: train/test dataset sizes and paths;
- `stage4_model_evaluation`: RMSE, MAE, and R2 by model;
- `stage4_best_model`: best model metadata and parameters;
- `stage4_model_prediction_samples`: prediction examples.

External Hive tables were selected because the data already lives in HDFS. Dropping or recreating an external table does not delete the underlying files, which is safer for a project pipeline where stages are rerun many times.

10 Data Analysis

We performed most of the EDA using Spark DataFrame operations. Spark was chosen because the raw trip table contains more than 22 million rows and because the later ML stage also uses Spark MLlib. Using Spark for both EDA and ML kept the processing environment consistent.

The analysis had three goals. First, we measured data quality through file audits, table profiling, column types, missingness, and numeric summaries. Second, we analyzed fuel-related behavior by vehicle type, speed bin, driving mode, temperature, HVAC usage, engine displacement, and geographical activity. Third, we produced a trip-level feature table for ML.

The main analytical questions were:

- What is the size and structure of the raw data?
- Which fields are missing and how does this affect fuel analysis?
- Which vehicle types dominate the dataset?
- How does estimated fuel rate differ by vehicle type?
- Does fuel rate increase with speed?
- How do stop-go, city, normal, and highway modes differ?
- How are outside temperature, HVAC usage, and fuel behavior related?
- Which trip-level features are suitable for ML modeling?

11 Analysis Results

The most important analysis results are:

1. **The raw telemetry table is large.** The `trips` table contains 22.4 million sensor-level records, while the `vehicles` table contains 384 vehicle metadata records.
2. **The raw table is not ML-ready.** Since one trip consists of many telemetry rows, the raw `trips` table must be aggregated before trip-level forecasting.
3. **Fuel-related missingness is the main limitation.** Direct `fuel_rate` is missing for about 96% of rows, so a fallback fuel-rate signal is required.
4. **MAF is a practical fallback.** MAF is much more available than direct fuel rate, so we used it to approximate fuel-rate intensity when direct measurements were unavailable.
5. **Vehicle type matters.** ICE and HEV vehicles have different fuel behavior in our results, so vehicle type is kept as a categorical ML feature.
6. **Speed and driving mode matter.** In our aggregated results, fuel rate generally grows across speed bins and differs between stop-go, city, normal, and highway regimes.
7. **Temperature and HVAC usage provide context.** Heater usage is more relevant at low outside temperatures, while air-conditioning usage increases at higher temperatures.
8. **Trip-level modeling is feasible.** After aggregation and filtering, the dashboard reports about 25.2k trainable trips.

12 Charts

This section presents the main analytical charts used in the report and dashboard.

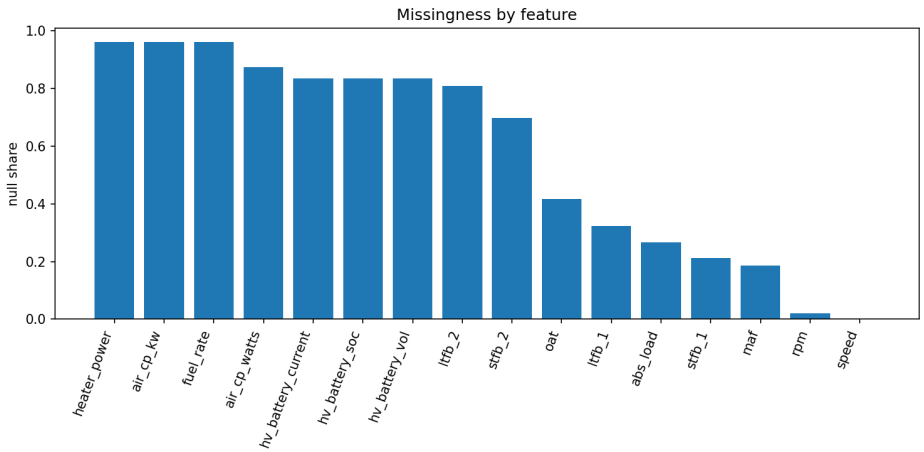

Figure 3: Missingness by feature. Fuel, HVAC, and hybrid battery fields are the sparsest groups.

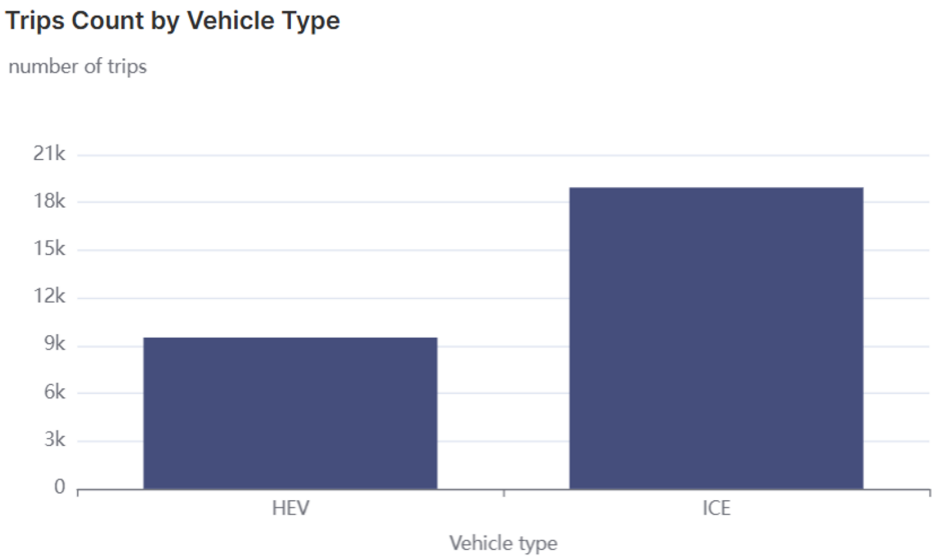

Figure 4: Vehicle type distribution. ICE vehicles dominate the dataset, while HEV is smaller group.

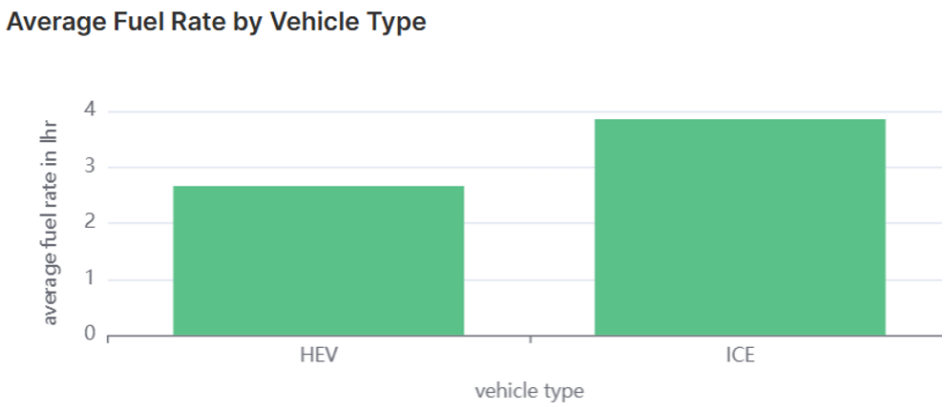

Figure 5: Fuel rate by vehicle type. Vehicle metadata is useful because average fuel-rate behavior differs by vehicle type.

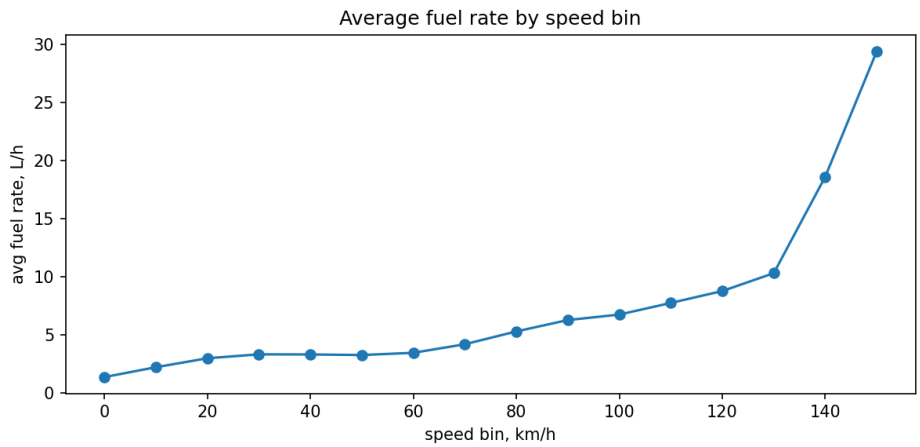

Figure 6: Fuel rate by speed bin. Average fuel rate increases strongly in higher-speed bins.

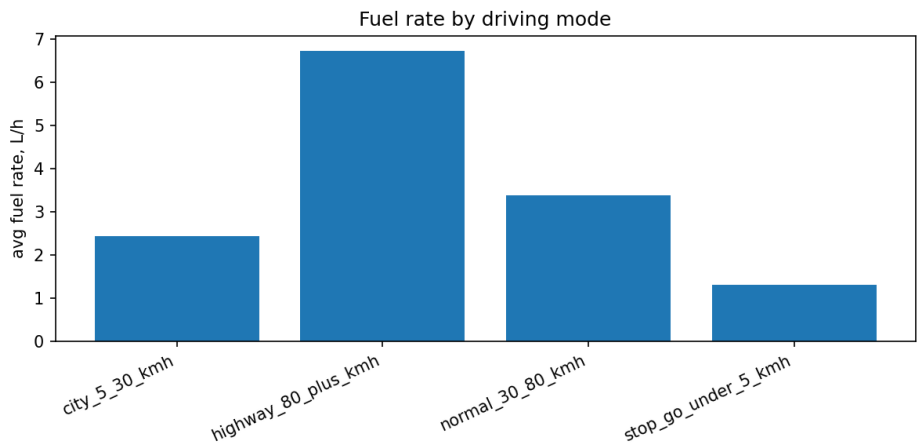

Figure 7: Fuel rate by driving mode. Stop-go, city, normal, and highway regimes have different fuel-intensity profiles.

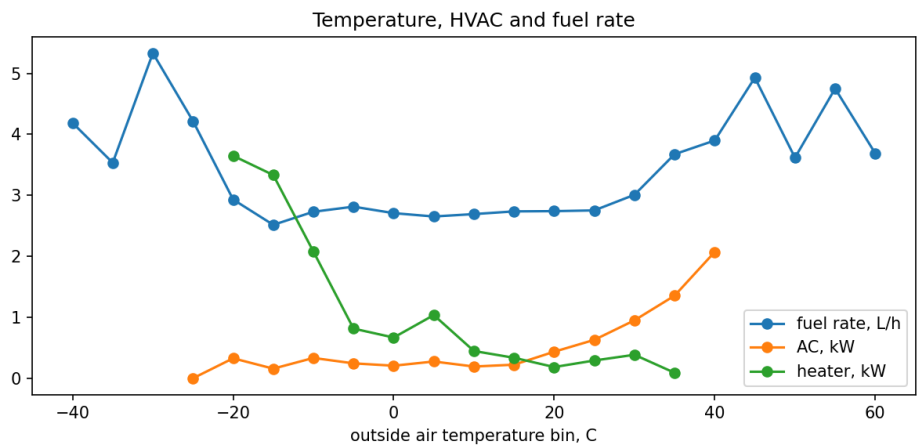

Figure 8: Temperature, HVAC usage, and fuel rate. HVAC behavior changes with outside temperature.

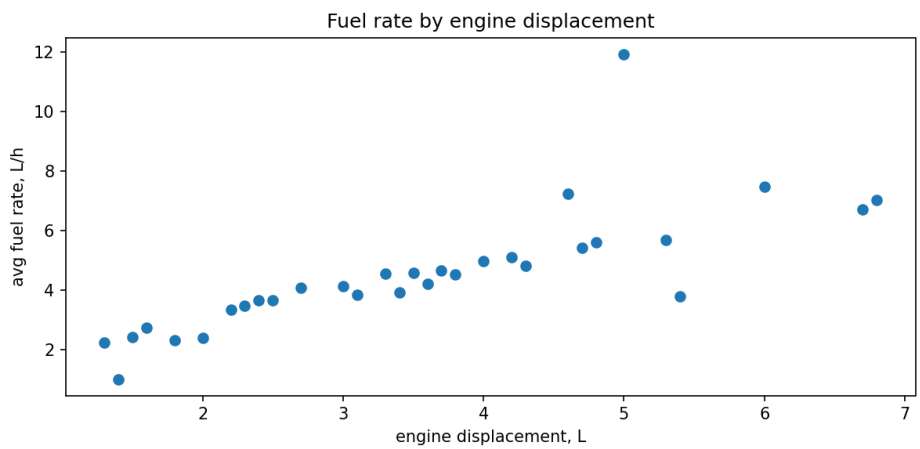

Figure 9: Engine displacement and fuel rate. Larger engines tend to have higher fuel rate, but the relationship is affected by driving conditions and vehicle type.

13 Interpretation

From the EDA, we can see that fuel consumption is related to both dynamic trip behavior and static vehicle characteristics. Dynamic behavior includes speed profile, stop-go time, idle time, RPM, MAF, and engine load. Static characteristics include vehicle type, weight, engine type, engine displacement, transmission, and drivetrain.

The missingness results explain why we used a MAF-based approximation. Without this approximation, direct fuel-rate analysis would cover only a very small part of the raw telemetry. At the same time, this approximation is still an engineering estimate, so the report and dashboard treat it carefully and use filtering to reduce unrealistic values.

The speed-bin and driving-mode charts were one of the reasons why we decided to test nonlinear models. Fuel consumption does not increase in a perfectly linear way with

speed. It can change more sharply at higher speed regimes and can interact with engine load and vehicle type. For this reason, we included tree-based models such as Random Forest and Gradient-Boosted Trees in addition to Linear Regression.

We divided the dashboard into Data Description, EDA, and PDA tabs because this order is easier to follow. It first explains what data we have, then shows the main analytical insights, and finally presents predictive modeling results.

14 Machine Learning Modeling

We formulated the predictive task as a regression problem. The model predicts `fuel_l_per_100km` from trip-level behavior, vehicle metadata, and environmental variables.

14.1 Feature Extraction and Preprocessing

The ML input is the Stage 2 trip-level feature table. Stage 3 performs feature selection and creates train/test Parquet datasets. The numeric candidate features include trip duration, observed seconds, distance, speed statistics, stop-go ratio, idle time, MAF statistics, RPM statistics, engine-load mean, outside temperature, hybrid battery statistics, vehicle weight, and engine displacement.

We used `SelectKBest` with `mutual_info_regression` and kept the top 10 numeric features. We chose mutual information because it can capture nonlinear relationships between numeric features and the regression target. This is more suitable than simple linear correlation when fuel consumption may depend on nonlinear patterns. Categorical vehicle metadata is kept separately because features such as vehicle type and engine configuration are important for interpretation and modeling.

Before the split, the pipeline removes invalid target values, filters approximate 1st and 99th percentile outliers, keeps trips with `distance_km >= 0.5`, removes invalid durations, and keeps realistic average speeds. The data is split into 70% training and 30% testing with random seed 42.

The Spark ML preprocessing pipeline uses:

- median imputation for numeric columns, because median is more robust to outliers than mean;
- `StringIndexer(handleInvalid="keep")` for categorical variables, because unseen or missing categories should not break prediction;
- one-hot encoding for categorical variables, because Spark ML models require numeric feature vectors;
- vector assembly to create the final feature vector;
- standard scaling for Linear Regression only, because linear models are sensitive to feature scale, while tree-based models are not.

14.2 Training and Fine-Tuning

The Stage 3 pipeline trains three Spark ML regression models:

- **Linear Regression**: used as a fast and interpretable baseline;
- **Random Forest Regressor**: used because it handles nonlinear feature interactions and is robust for tabular data;
- **Gradient-Boosted Trees Regressor**: used because boosting can capture complex nonlinear patterns and often performs well on structured regression tasks.

All models are trained in Spark on YARN, not as local-only scripts. Hyperparameter tuning is done with Spark `CrossValidator` and `ParamGridBuilder`. We used three-fold cross-validation with RMSE as the optimization metric. RMSE was selected for tuning because it penalizes larger prediction errors more strongly, which is useful for fuel-consumption forecasting where large errors are more harmful.

Table 6: Hyperparameter grids used in the Spark ML pipeline.

Model	Parameter	Values
Linear Regression	`regParam`	0.0, 0.01, 0.1
Linear Regression	`elasticNetParam`	0.0, 0.5, 1.0
Linear Regression	`aggregationDepth`	2, 3, 4
Random Forest Regressor	`maxDepth`	4, 6, 8
Random Forest Regressor	`numTrees`	20, 40, 60
Random Forest Regressor	`subsamplingRate`	0.7, 0.85, 1.0
GBT Regressor	`maxDepth`	3, 4, 5
GBT Regressor	`maxBins`	32, 64, 128
GBT Regressor	`stepSize`	0.03, 0.05, 0.1

14.3 Evaluation

The models are evaluated on the held-out test set with RMSE, MAE, and R2. RMSE and MAE are both reported because RMSE highlights large errors while MAE is easier to interpret as the average absolute error in L/100km. R2 shows how much variation in the target is explained by the model.

Table 7: Evaluation of the best models on the test set.

Model	RMSE	MAE	R2
Linear Regression	1.9562	1.0059	0.7804
Random Forest Regressor	1.3968	0.7564	0.8882
GBT Regressor	1.4696	0.7656	0.8762

The **RandomForestRegressor** turned out to be the best-performing model. It achieved the lowest errors among all models, and the highest coefficient of determination R2. The optimal configuration consisted of 60 trees (numTrees = 60) and a maximum depth of 8 (maxDepth = 8). These results significantly outperform linear regression and gradient boosted trees.

This result shows that the current best model can make reasonably accurate predictions on the cleaned trip-level dataset. However, the result should be interpreted with the limitations of the dataset in mind. Fuel consumption depends on driving conditions, road context, weather, vehicle state, and possible sensor noise. Some of these factors are not fully represented in the current dataset.

15 Data Presentation

15.1 Dashboard Description

The final presentation layer is an Apache Superset dashboard. The dashboard is published at:

http://hadoop-03.uni.innopolis.ru:8808/superset/dashboard/221/

The dashboard was divided into three tabs:

- **Data Description**: raw table scale, PostgreSQL table records, table schemas, sample rows, and practical data recommendations;
- **EDA**: data-quality charts and analytical insights about fuel behavior;
- **PDA**: machine learning conclusion, best model configuration, model error comparison, R2 comparison, and selected features.

We organized the dashboard in this order because it is easier to understand the project step by step: first the user sees what data we have, then the main EDA results, and then the predictive modeling results.

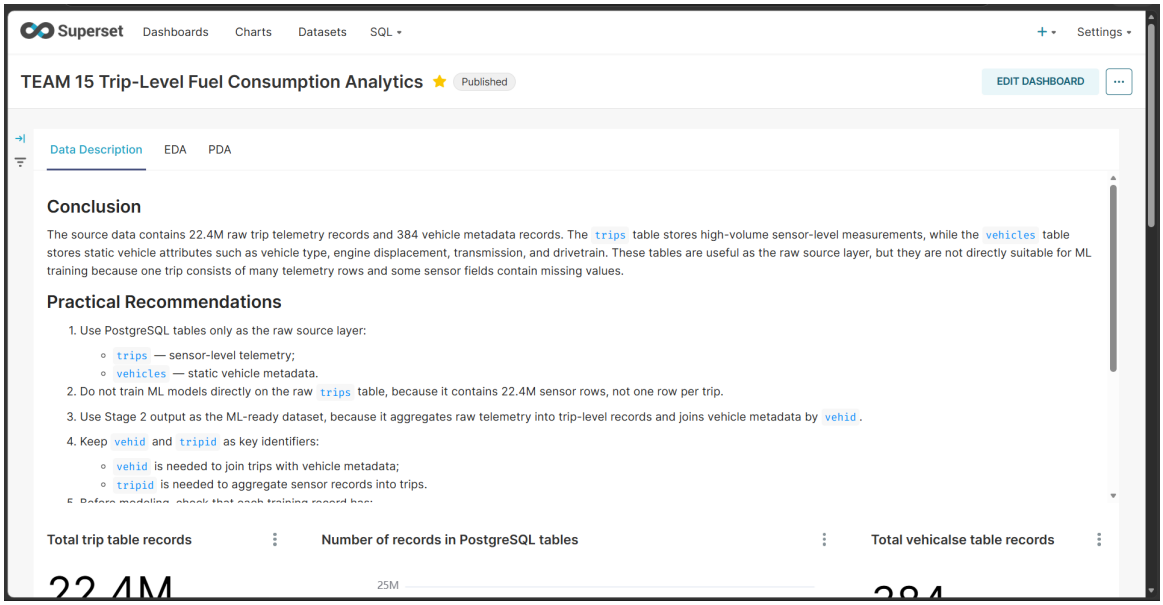

Figure 10: Data Description tab: conclusion and practical recommendations about raw PostgreSQL tables.

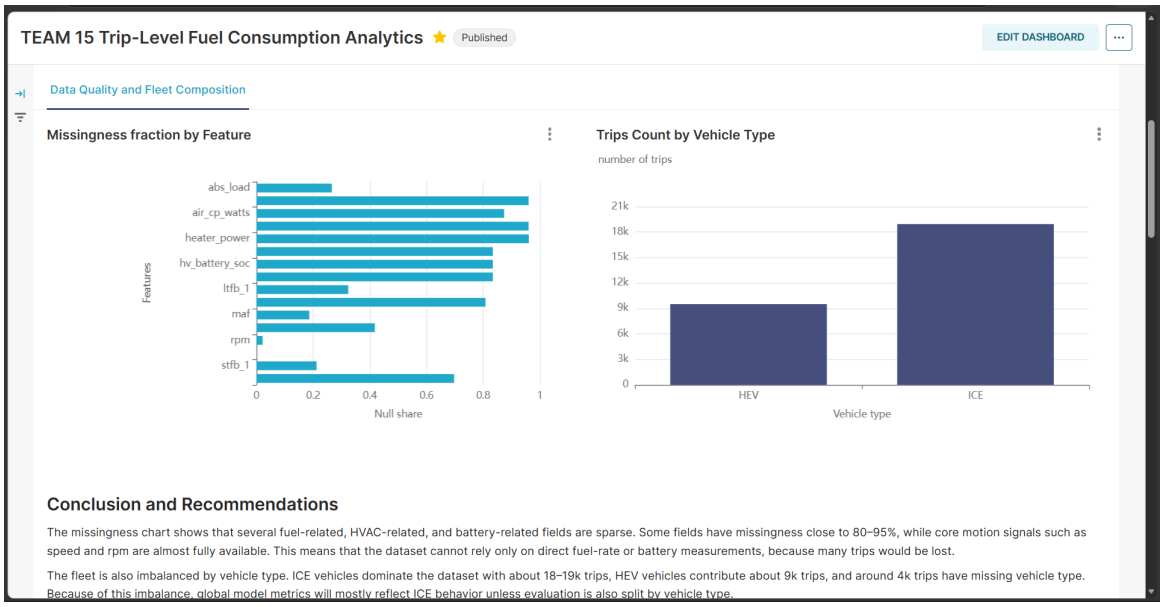

Figure 11: EDA tab: missingness and vehicle-type distribution.

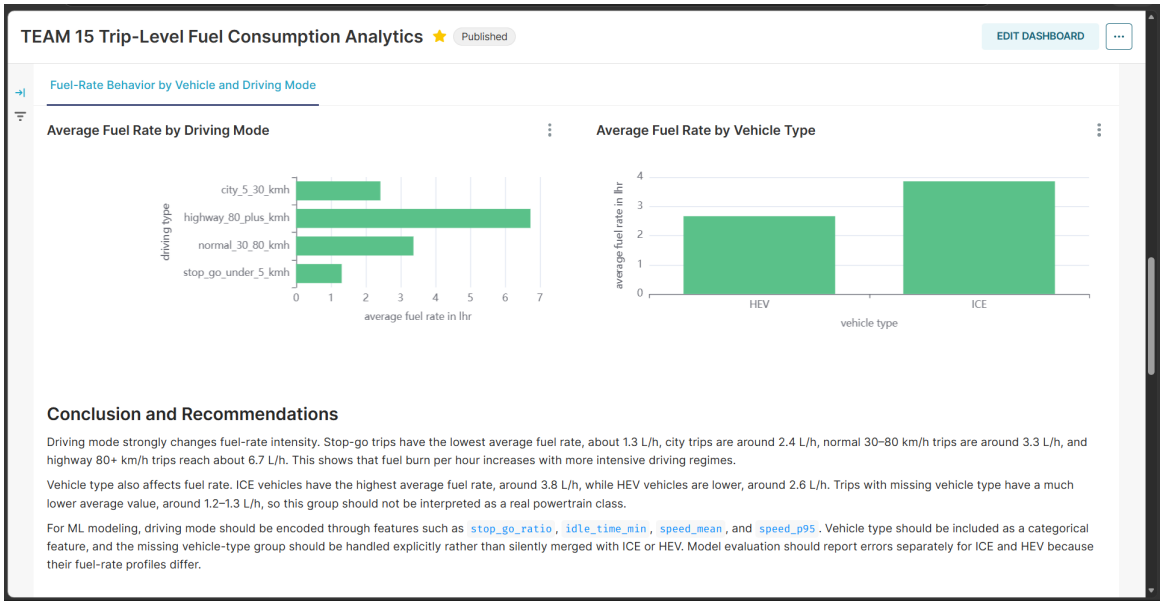

Figure 12: EDA tab: fuel-rate behavior by driving mode and vehicle type.

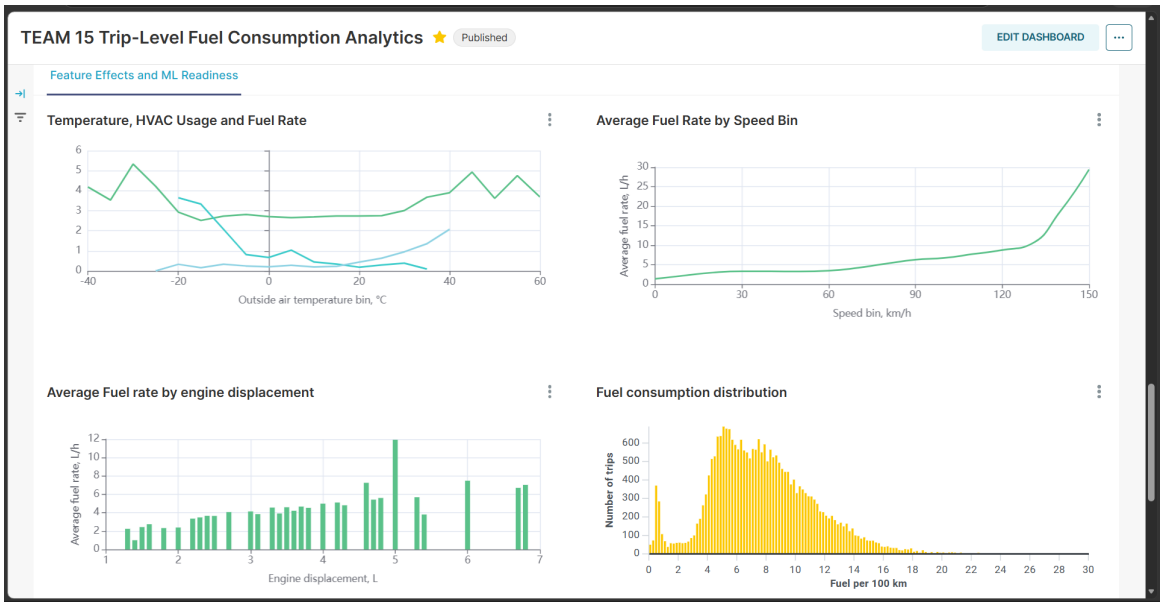

Figure 13: EDA tab: temperature/HVAC, speed bins, engine displacement, and fuel-consumption distribution.

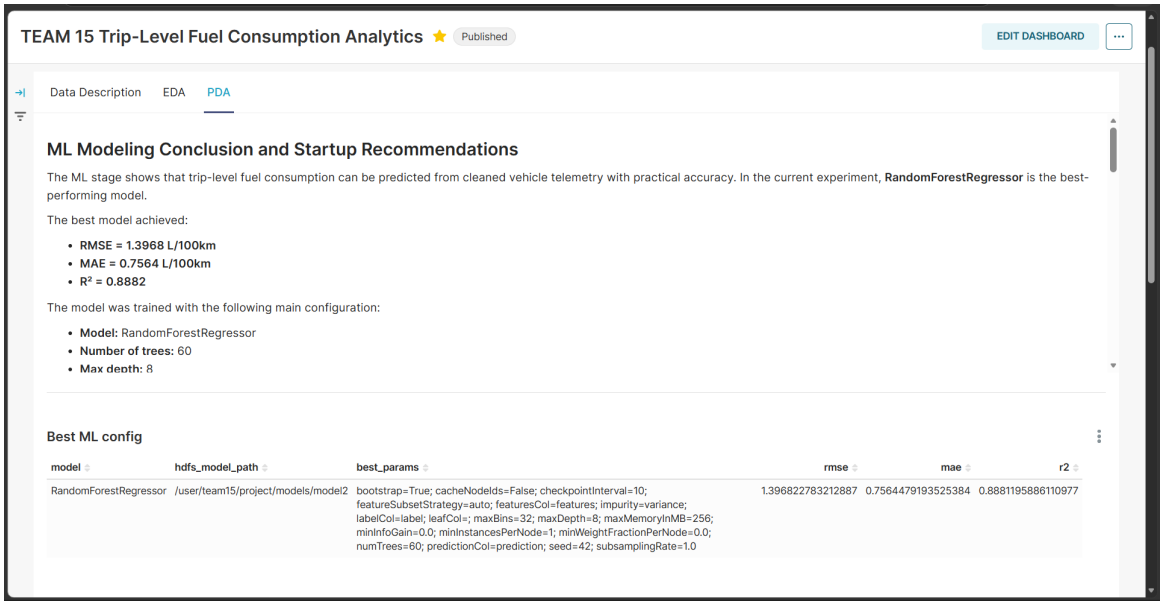

Figure 14: PDA tab: ML conclusion and best Random Forest configuration.

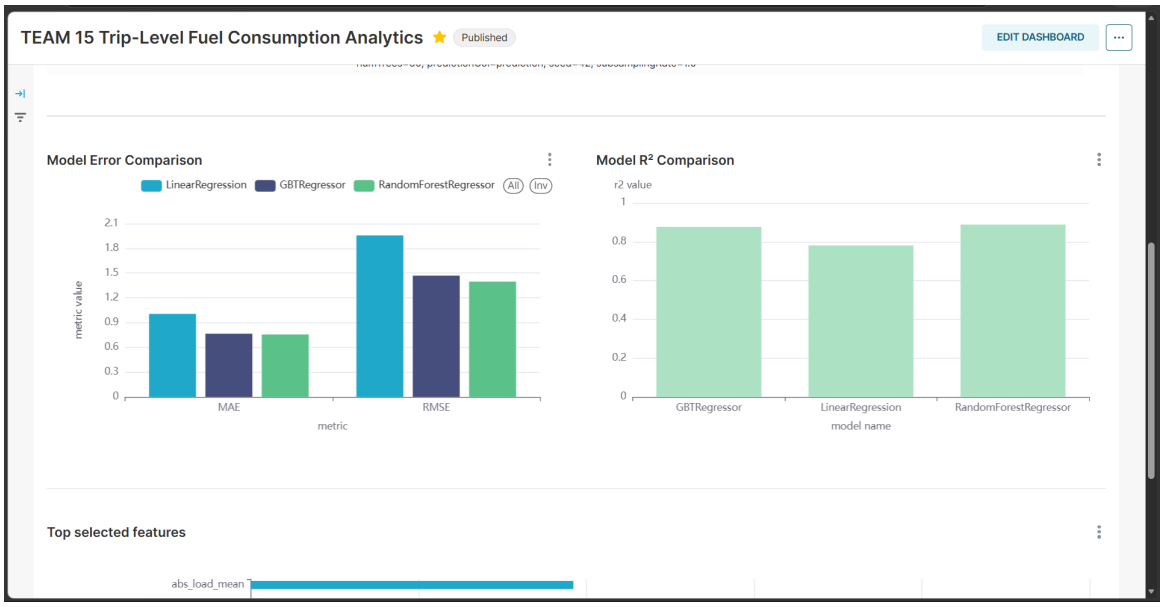

Figure 15: PDA tab: model error and R2 comparison.

15.2 Charts Description

Table 8: Dashboard chart descriptions.

Chart / block	Purpose	Interpretation
Total raw records and vehicle records	Shows the scale of the source layer	Raw data is large and sensor-level, while vehicle metadata is small and static.

Chart / block	Purpose	Interpretation
PostgreSQL table schemas	Shows table structure and datatypes	The source layer has separate `trips` and `vehicles` tables.
Sample rows	Gives concrete examples from source tables	The raw `trips` table contains many rows per trip and should not be used directly as one row per training example.
Missingness by feature	Shows data-quality limitations	Fuel, HVAC, and hybrid battery fields require careful handling.
Vehicle type distribution	Shows fleet composition	ICE vehicles dominate, so model interpretation should consider imbalance.
Fuel rate by driving mode	Compares fuel intensity across speed regimes	Higher-speed modes usually have higher fuel-rate intensity.
Fuel rate by vehicle type	Compares powertrain groups	ICE and HEV vehicles behave differently, so vehicle type is a useful feature.
Temperature, HVAC, and fuel	Shows environmental context	HVAC behavior changes with temperature and may influence consumption.
Fuel rate by speed bin	Shows speed effect	Fuel intensity rises in high-speed bins.
Engine displacement and fuel rate	Shows vehicle design effect	Larger engines tend to have higher fuel rate, but the relation is not purely linear.
Model error comparison	Compares RMSE and MAE	Tree-based models reduce error compared with the linear baseline.
Model R2 comparison	Compares explained variance	The best Random Forest model has the highest R2 in the dashboard.
Top selected features	Shows selected predictors	Engine load, speed, MAF, distance, and trip behavior are important ML inputs.

16 Findings

The main findings of the project are:

1. The raw VED telemetry data is large enough to require distributed processing, with 22.4 million sensor-level trip records.
2. The raw `trips` table is not directly suitable for ML because it contains many rows per trip; trip-level aggregation is required.
3. MAF-based fuel-rate approximation is a justified practical solution because MAF is more available than direct fuel rate.
4. In our analysis, fuel behavior was strongly related to speed and driving mode.
5. ICE and HEV vehicles show different average fuel-rate behavior, so vehicle type should be included in analysis and modeling.
6. The ML pipeline shows that trip-level fuel consumption can be predicted with practical accuracy from cleaned telemetry.
7. RandomForestRegressor is the best model in the dashboard, with RMSE 1.3968 L/100km, MAE 0.7564 L/100km, and R2 0.8882.

17 Conclusion

In this project, we implemented a distributed pipeline for trip-level fuel consumption analytics and forecasting. The pipeline starts with raw VED telemetry files and vehicle metadata, loads them into PostgreSQL, imports them into HDFS through Sqoop, processes them with Spark, stores analytical outputs in Parquet, trains Spark ML models, and presents the results in Superset.

A key result is that raw telemetry must be transformed before it can be useful for ML. One trip is spread across many timestamp-level records, and direct fuel-rate values are highly sparse. For this reason, the project uses vehicle metadata enrichment, MAF-based fuel-rate approximation, trip-level aggregation, feature selection, and train/test filtering.

The EDA showed that fuel consumption is related to driving behavior, speed, vehicle type, engine displacement, temperature, and HVAC usage. The PDA stage showed that nonlinear tree-based models are more suitable than a simple linear baseline for this tabular telemetry task. In the dashboard, RandomForestRegressor is the best-performing model.

18 Summary of the Report

This report follows the required project structure and summarizes all main parts of our work. The report explains why we selected the main technical methods: PostgreSQL for relational source storage, Sqoop for bulk import to HDFS, Parquet + Snappy for read-heavy columnar analytics, Spark for distributed EDA and ML, MAF-based estimation for

sparse fuel-rate data, trip-level aggregation for ML readiness, mutual-information feature selection for nonlinear relevance, and Superset for dashboard presentation.

19 Reflections on Own Work

During the project, we understood that a big data project is not only about model training. A large part of the work was spent on building a reliable data path: downloading and preparing data, designing tables, importing to HDFS, choosing file formats, profiling data quality, creating analytical outputs, validating ML inputs, and preparing dashboard tables.

We also learned that modeling results depend strongly on earlier engineering decisions. If raw sensor rows were used directly, the model would not represent trip-level behavior correctly. If leakage fields were included, the reported model quality could become misleading.

20 Challenges and Difficulties

Table 9: Main project challenges and solutions.

Challenge	**Why it was a problem**	**Solution**
Choosing storage format	Different formats are optimized for different workloads	Chose Parquet + Snappy because the project is read-heavy and column-oriented
Pipeline reproducibility	Stage scripts should be rerunnable without stale outputs causing failures	Scripts remove old outputs before creating new ones where needed
Model leakage risk	Some fuel-derived fields are too close to the target	Excluded leakage-prone fields from model features
Feature dimensionality	Many candidate features can make modeling harder and dashboard explanation less clear	Used mutual-information feature selection and kept categorical metadata

21 Recommendations

For future work, we recommend:

- monitoring model quality separately for ICE and HEV vehicles;
- adding external context such as road type, elevation, traffic, and weather;

- improving outlier detection for short trips and unusual fuel-consumption values.

22 Table of Contributions

Task	Description	Mikhail	Ilyas	Andrei	Kamilya	Deliverables	Hours
Data Extraction	Collect the raw vehicle telemetry dataset and prepare source files for ingestion	80%	20%	0%	0%	Raw dataset files, data collection scripts	6
Data Cleaning	Handle missing values, invalid records, inconsistent fuel-related fields, and prepare usable records	10%	50%	40%	0%	Cleaned trip-level records, trainable subset	8
Data Transformation	Transform raw files into structured database and distributed storage format	70%	15%	15%	0%	PostgreSQL tables, HDFS Parquet files	8
Data Analysis	Perform exploratory data analysis, study feature distributions, and identify fuel-consumption patterns	0%	80%	0%	20%	EDA queries, analytical outputs, data insights	10
Feature Engineering	Create and select meaningful features for trip-level fuel-consumption forecasting	0%	10%	90%	0%	ML-ready feature dataset, feature pipeline	9
Model Development	Train and tune distributed Spark ML models for fuel-consumption forecasting	0%	20%	80%	0%	Linear Regression, Random Forest, GBT models	10
Evaluation	Evaluate model performance using regression metrics and compare model quality	0%	20%	80%	0%	Evaluation tables, model comparison results	6

Visualization	Create charts for data quality, vehicle composition, fuel behavior, and feature effects	0%	80%	0%	20%	EDA charts, dashboard visualizations	8
Dashboard	Develop and organize the Apache Superset dashboard with storytelling structure	0%	70%	0%	30%	Published Superset dashboard	7
Report Writing	Document all project stages, organize the final report, describe repository structure, and check artifacts	5%	5%	0%	90%	Final report, README files, project documentation	12